

[CS230] Structuring Machine Learning Projects: Satisficing and Optimizing Metrics

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-5. Deep Learning Fundamentals - *Completed*

Chapter 6. Structuring ML Projects (Current Unit)

- 6.1 Orthogonalization Strategy - *Completed*
- 6.2 Single Number Evaluation Metric - *Completed*
- **6.3 Satisficing and Optimizing Metrics**
 - * The Dilemma: Accuracy vs Latency
 - * Definition: Optimization vs Constraint
 - * The N -Metric Rule
 - * Implementation: Filtering Logic
- 6.4 Human-level Performance - *Upcoming*

지난 시간 복습 및 연결

지난 시간에 우리는 정밀도와 재현율을 F1 Score로 합치는 법을 배웠습니다. 그런데 현실 문제는 더 복잡합니다. 모델 A: 정확도 99%, 응답시간 1.5초 모델 B: 정확도 98%, 응답시간 0.05초 A는 성능은 좋지만 1.5초나 걸려서 아무도 안 쓸 겁니다. 그렇다고 정확도와 시간을 평균 낼 수도 없습니다(단위가 다름). 이런 딜레마를 해결하기 위해 앤드류 응 교수는 "만족(Satisficing) 지표와 최적화(Optimizing) 지표의 분리"를 제안합니다.

1 Unit Overview

□ 핵심 목표

이 단원은 여러 개의 목표가 충돌할 때, 우선순위를 정하는 전략적 프레임워크를 다룹니다.

- 구분: 평가 지표를 '최대한 좋게 할 것(Optimizing)'과 '기준만 넘기면 되는 것(Satisficing)'으로 나눕니다.
- 규칙: N 개의 지표가 있다면, 1개만 최적화하고 나머지 $N - 1$ 개는 제약 조건으로 둡니다.
- 구현: 여러 모델 후보 중 최적의 모델을 자동으로 선별하는 파이썬 코드를 작성합니다.

2 Essential Terminology

구분	정의	예시
Optimizing Metric	다다익선. 무한히 좋아질수록 좋은 단 하나의 지표.	정확도(Accuracy), F1 Score
Satisficing Metric	임계값(Threshold)만 넘으면 통과(Pass).	실행 시간(Latency) $\leq 100ms$

3 Core Concepts: 올림픽 달리기

3.1 1. The Rule of N Metrics

고려해야 할 지표가 3개(정확도, 속도, 메모리)라면 어떻게 해야 할까요? ”3마리 토끼를 다 잡으려다가는 다 놓칩니다.”

□ 올림픽 달리기와 도핑 테스트 (직관적 비유)

- **Optimizing** (달리기 기록): 0.01초라도 빠르면 무조건 좋습니다. 금메달의 기준입니다.
- **Satisficing** (도핑 테스트): 약물이 ”아주 조금 검출됨”이나 ”전혀 검출 안 됨”이나 똑같이 통과(Pass)입니다. 기준치만 안 넘으면 됩니다. 더 깨끗하다고 가산점을 주진 않습니다.

3.2 2. Mathematical Formulation (수학적 정의)

이 전략은 머신러닝 문제를 ’제약 조건이 있는 최적화 문제’로 바꿉니다.

$$\begin{aligned} &\text{Maximize } \mathbf{Accuracy} \\ &\text{subject to } \mathbf{Latency} \leq 100ms \\ &\text{and } \mathbf{Memory} \leq 500MB \end{aligned}$$

- **Accuracy:** 목적 함수 (Objective Function) $\rightarrow$ Optimizing
- **Latency, Memory:** 제약 조건 (Constraints) $\rightarrow$ Satisficing

가중치 합의 함정 (오해 방지 가이드)

”그냥 $\text{Score} = \text{Accuracy} - 0.5 \times \text{Latency}$ 처럼 합치면 안 되나요?” **비추천합니다.** 1. 단위가 다릅니다(Accuracy는 %, Latency는 ms). 2. 사용자 경험은 비선형적입니다. 100ms가 넘으면 ’렉 걸림’을 느껴 바로 앱을 끕니다. 이를 선형 식으로 표현하기 어렵습니다. Satisficing(Cut-off)이 훨씬 자연스럽습니다.

4 Implementation: Model Selector

여러 모델의 성능표를 입력받아 자동으로 최적 모델을 뽑는 로직을 구현합니다.

```

1 class ModelSelector:
2     def __init__(self, optimize_key, constraints):
3         """
4         optimize_key: 최적화할 지표 예(: 'accuracy')
5         constraints: 지표명{: 임계값(, 연산자)}
6                     예: {'latency': (100, 'lt')} -> less than 100
7         """
8         self.opt_key = optimize_key
9         self.constraints = constraints
10
11     def select(self, models):
12         # 1. Satisficing 단계 (Filtering)
13         valid_models = []
14         for m in models:
15             is_valid = True
16             for key, (thresh, op) in self.constraints.items():
17                 val = m[key]
18                 if op == 'lt' and val > thresh: is_valid = False
19                 if op == 'gt' and val < thresh: is_valid = False
20
21             if is_valid:
22                 valid_models.append(m)
23
24         if not valid_models:
25             print("No models satisfied constraints!")
26             return None
27
28         # 2. Optimizing 단계 (Sorting)
29         # 점수 높은 순으로 정렬
30         best_model = sorted(valid_models, key=lambda x: x[self.opt_key], reverse=
31                             True)[0]
32         return best_model
33
34 # --- 실행 ---
35 if __name__ == "__main__":
36     candidates = [
37         {'id': 'A', 'acc': 0.99, 'lat': 1500}, # 성능 굿, 너무 느림
38         {'id': 'B', 'acc': 0.98, 'lat': 90},   # 성능 적당, 빠름 (Pass)
39         {'id': 'C', 'acc': 0.90, 'lat': 50},   # 너무 빠름, 성능 별로 (Pass)
40         {'id': 'D', 'acc': 0.985, 'lat': 110}, # 아깝게 느림 (Fail)
41     ]
42
43     # 전략: Latency < 100ms 인 것 중에서 Accuracy 최대화
44     constraints = {'lat': (100, 'lt')}
45     selector = ModelSelector('acc', constraints)
46
47     best = selector.select(candidates)
48     print(f"Best Model: {best['id']} (Acc: {best['acc']}, Lat: {best['lat']})")
49     # A, D 탈락. B(0.98) vs C(0.90) -> B 승리.

```

Listing 1: Auto Model Selector Logic

5 FAQ & Pitfalls

Q. 만족 지표(Satisficing)가 너무 뻑뻑하면 어떡하나요?

A. 만약 ‘Latency < 10ms’로 걸었는데 통과하는 모델이 하나도 없다면, 하드웨어를 바꾸거나 비즈니스 요구사항(임계값)을 완화해야 합니다.

Q. 최적화 지표를 2개로 하면 안 되나요?

A. 안 됩니다. ”정확도도 높고 속도도 빠른 것”을 찾으려 하면, A(정확도짱)와 B(속도짱) 사이

에서 결정을 못 내립니다. 결국 둘을 합친 단일 지표를 만들거나, 하나를 제약 조건으로 돌려야 합니다.

다음 단계 (Next Step)

이제 평가 기준(Metric)까지 완벽하게 세팅했습니다. 모델을 열심히 개선하고 있는데, 문득 의문이 듭니다. ”도대체 이 모델은 어디까지 좋아질 수 있는 걸까? 100%가 가능한가?”

이 질문에 답하기 위해서는 비교 대상, 즉 기준점(Baseline)이 필요합니다. 다음 시간에는 [Human-level Performance]를 통해, 나의 모델이 현재 어느 수준인지, 그리고 얼마나 더 발전할 여지가 있는지(Bayes Error) 가늠하는 법을 배웁니다.

□ 단원 요약 (Cheat Sheet)

1. **Optimizing:** 최대한 좋게 만들어야 하는 단 하나의 지표 (예: Accuracy).
2. **Satisficing:** 기준선(Threshold)만 넘으면 되는 지표들 (예: Latency, Cost).
3. **Process:** Satisficing 지표로 필터링(Cut)하고, Optimizing 지표로 줄 세운다(Rank).
4. **Rule:** N 개의 지표가 있다면 1개는 Optimizing, $N - 1$ 개는 Satisficing이다.